

M4 Phase 3: Deployment Validation

Hosting Platform Decision Report

DeepMindQ CRM | Technical Architecture Assessment

August 6, 2026

Executive Summary

This report evaluates three deployment strategies to resolve the Vercel Hobby plan limitation of 12 serverless functions per deployment, which blocks the deployment of DeepMindQ CRM with its 250 individual API route files. After extensive technical analysis of each option, including cost modeling, platform capability assessment, and architectural impact evaluation, this document provides a clear recommendation with supporting evidence and trade-off analysis.

The core finding is that the 12-function limit is not an application architecture defect but a platform tier constraint. DeepMindQ CRM follows standard Next.js App Router conventions with one route handler per API endpoint, which is the recommended pattern for maintainability, testability, and developer experience. The application generates 250 serverless functions from 79 top-level API directories, with the largest modules being AI operations (35 routes), intelligence services (34 routes), and company management (24 routes). This is a natural consequence of building a comprehensive enterprise CRM with dedicated endpoints for each business domain.

Application Metric	Value	Significance
Total API Route Files	250	Serverless functions generated on Vercel
Top-Level API Directories	79	Business domain organization units
Largest Module (AI)	35 routes	AI operations and model inference
Second Largest (Intelligence)	34 routes	Business intelligence services
Dynamic Route Segments	32	Parameterized URL patterns
Vercel Hobby Function Limit	12	Hard cap blocking deployment

Table 1: Application Metrics Overview

Recommendation Preview: Option 1 (Vercel Pro Upgrade) is the recommended path forward for the immediate resolution of the deployment blocker. It requires zero architectural changes, preserves the existing CI/CD pipeline investment, and provides unlimited serverless functions at a predictable \$20/seat/month base cost.

For long-term strategic flexibility, Option 2 (Container-Based Deployment on Azure Container Apps or AWS ECS/Fargate) should be evaluated as a Phase 5 infrastructure optimization initiative.

Table of Contents

- Executive Summary** 1
- 1. Current Architecture Analysis** 5
 - 1.1 Application Architecture Overview 5
 - 1.2 API Route Distribution 5
 - 1.3 Platform Constraint Analysis 6
- 2. Option 1: Vercel Pro Upgrade** 7
 - 2.1 Plan Overview and Pricing 7
 - 2.2 Cost Impact Analysis 8
 - 2.3 Architectural Compatibility 8
 - 2.4 Advantages and Risks 9
- 3. Option 2: Alternative Hosting Platforms** 10
 - 3.1 Evaluation Criteria 10
 - 3.2 Platform Comparison Matrix 10
 - 3.3 Platform Deep Dives 11
 - 3.3.1 AWS ECS/Fargate** 11
 - 3.3.2 Azure Container Apps** 11
 - 3.3.3 Railway** 12
 - 3.3.4 Render** 12
 - 3.3.5 Fly.io** 12
 - 3.4 Decision Matrix Summary 13
- 4. Option 3: API Architecture Refactor** 14
 - 4.1 Refactor Approach Analysis 14
 - 4.2 Route Grouping Assessment 14
 - 4.3 Impact Assessment 15
 - 4.4 Migration Risk 15
- 5. Recommendation** 16
 - 5.1 Primary Recommendation: Vercel Pro (Option 1) 16
 - 5.2 Implementation Path 17
 - 5.3 Long-Term Strategic Consideration 17

5.4 Options NOT Recommended17

1. Current Architecture Analysis

1.1 Application Architecture Overview

DeepMindQ CRM is built on Next.js 16.1.1 with React 19, using the App Router architecture pattern introduced in Next.js 13 and matured through subsequent releases. The application follows the standard convention of placing API route handlers in files named **route.ts** within the **src/app/api/** directory structure. Each file exports HTTP method handlers (GET, POST, PUT, DELETE, PATCH) that Vercel compiles into individual serverless functions. This is the officially recommended pattern by the Next.js and Vercel teams for building production applications, providing clear separation of concerns, independent deployability, and optimal cold-start performance per endpoint.

The database layer uses Prisma ORM (version 6.19.3) connecting to Neon PostgreSQL, with both pooled and direct connection URLs configured for runtime and migration operations respectively. The application integrates Sentry (version 10.67.0) for error monitoring and performance tracking. Authentication helpers are implemented in dedicated utility modules under **src/lib/auth-helpers.ts**, though a **middleware.ts** file for Edge-level request interception is currently absent despite being referenced in configuration files. The application is deployed to the Mumbai region (bom1) on Vercel with a daily cron job configured for batch processing at 06:00 UTC.

1.2 API Route Distribution

The 250 API routes are distributed across 79 top-level directories, reflecting the breadth of the CRM platform. The following table shows the top 15 modules by route count, which together account for approximately 72% of all API routes. This distribution reveals a well-organized domain-driven architecture with dedicated modules for AI capabilities, business intelligence, core CRM operations (companies, leads, contacts), and specialized features such as email sequences, health monitoring, and acquisition intelligence.

Module	Routes	Domain	Description
ai/	35	AI Operations	Model inference, embeddings, chat completions
intelligence/	34	Business Intel	Competitive analysis, market signals
companies/	24	Core CRM	Company CRUD, relationships, enrichment
leads/	10	Pipeline Mgmt	Lead scoring, qualification, assignment
contacts/	9	Core CRM	Contact management, communication history

Module	Routes	Domain	Description
auth/	9	Security	Authentication, sessions, verification
sequences/	6	Automation	Email sequences, step management
health/	6	Observability	Health checks, system monitoring
g-intel-acquisition/	6	Intelligence	Acquisition target analysis
capabilities/	6	Platform	Feature flags and capability management
reports/	4	Analytics	Report generation and scheduling
recommendations/	4	AI-Assisted	AI-driven action recommendations
knowledge/	4	Knowledge Mgmt	Knowledge base and article management
engines/	4	Processing	Background processing engine control
signals/	3	Intelligence	Market signal detection and alerts

Table 2: Top 15 API Modules by Route Count

The remaining 45 top-level directories each contain a single route file, handling specialized functions such as email worker processing, data import/export, audit logging, analytics aggregation, compliance checks, dashboard aggregation, and administrative operations. There are 32 dynamic route segments (using the [param] convention) for parameterized URLs, enabling nested resource access patterns such as **companies/[id]/notes/[noteId]**. No catch-all route segments ([...catchAll] or [...catchAll])) currently exist, meaning every API endpoint has an explicitly defined handler, which is the safest pattern for type safety, OpenAPI documentation generation, and security auditing.

1.3 Platform Constraint Analysis

The Vercel Hobby plan imposes a hard limit of 12 serverless functions per deployment. This limit is a pricing-tier constraint, not a technical limitation of the platform itself. When DeepMindQ CRM attempts to deploy, Vercel compiles each route.ts file into a separate serverless function, resulting in 250 functions that exceed the Hobby cap. The deployment fails with the error: "No more than 12 Serverless Functions can be added to a Deployment on the Hobby plan. Create a team (Pro plan) to deploy more." This failure occurs at the build stage before any runtime execution, meaning the application is completely undeployable on the current plan.

It is critical to understand that this is not a code defect. The Next.js App Router pattern of one route.ts file per API endpoint is the standard, recommended approach used by thousands of production applications on Vercel. The 250 routes reflect the comprehensive scope of an enterprise CRM platform, not architectural bloat.

Consolidating routes into fewer handlers would reduce the function count but would simultaneously sacrifice the modularity, testability, and developer experience that the App Router pattern provides. Such a consolidation would be a significant architectural regression that introduces routing complexity, authentication granularity challenges, and testing difficulties.

2. Option 1: Vercel Pro Upgrade

2.1 Plan Overview and Pricing

Vercel Pro is the immediate next tier above the free Hobby plan. It is priced at **\$20 per developer seat per month**, with a \$20/month included spending credit that offsets usage-based costs for compute, bandwidth, and other billable resources. For a solo developer or small team with moderate traffic, the effective monthly cost often stays near the base \$20 seat fee, as the included credit covers typical usage patterns. The Pro plan removes the 12-function hard limit entirely, replacing it with unlimited serverless function creation per deployment, with costs driven only by actual execution volume.

The key architectural benefit is that **zero code changes are required**. The existing 250 API route files, Prisma configuration, environment variables, cron jobs, and CI/CD workflows continue to function without modification. The upgrade is purely an account-level plan change that takes effect on the next deployment. This preserves the entire M4 Phase 3 investment in pipeline stabilization, GitHub Actions workflows, secret management, and deployment automation.

Resource	Hobby (Free)	Pro (\$20/seat/mo)
Serverless Functions	12 (hard cap)	Unlimited
Function Invocations	1M/month	\$0.60 per 1M over included
Active CPU Time	4 CPU-hours included	\$0.128/hour over included
Max Function Duration	300 seconds	800 seconds (1800s beta)
Max Memory per Function	2 GB	4 GB
Bandwidth (Fast Data)	100 GB/month	1 TB/month included
Edge Requests	1M/month	10M/month included
Build Execution Hours	100 min/month	400 hours/month
Concurrent Builds	1	Up to 12
Concurrent Deployments	1	Up to 500
Deployments per Day	100	6,000

Resource	Hobby (Free)	Pro (\$20/seat/mo)
Environment Variables	1,000/env	1,000/env
Runtime Logs Retention	1 hour	1 day
SLA	None	99.9%

Table 3: Vercel Hobby vs Pro Feature Comparison

2.2 Cost Impact Analysis

The cost impact of Vercel Pro depends primarily on traffic volume and function execution patterns. For a CRM application with 250 API endpoints, most endpoints will experience low to moderate invocation rates, with higher traffic concentrated on core CRUD operations (companies, contacts, leads) and AI endpoints. The included \$20/month credit provides a meaningful buffer against usage charges, and for applications with fewer than 100,000 monthly invocations, the effective cost typically remains at the \$20 base fee.

Estimated monthly costs for DeepMindQ CRM under various traffic scenarios show that the Pro plan is highly cost-effective for the expected initial deployment phase. During early production with light user load, the total cost stays at \$20/month. As usage grows to moderate levels with several hundred thousand monthly invocations, costs rise to approximately \$25-40/month. Even under heavy production load approaching 1 million monthly invocations, the total remains under \$60/month, which is competitive with or cheaper than most container hosting alternatives when factoring in operational overhead.

Traffic Scenario	Monthly Invocations	Est. Total Cost	Notes
Development / Staging	< 10,000	\$20	Covered by included credit
Early Production	50,000 - 100,000	\$20 - \$25	Low-traffic CRM deployment
Moderate Production	200,000 - 500,000	\$25 - \$40	Growing user base
Heavy Production	1M+	\$40 - \$80	Full enterprise usage

Table 4: Estimated Monthly Costs by Traffic Scenario

2.3 Architectural Compatibility

The Vercel Pro upgrade offers perfect architectural compatibility with the existing application. Every aspect of the current deployment configuration remains

unchanged: the `vercel.json` build command, the Mumbai region setting, the GitHub auto-deploy integration, the cron job for batch processing, and the environment variable configuration all continue to work identically. The CI/CD pipeline built during M4 Phase 3, including the staging deployment workflow, production approval gate, database migration automation, and smoke test validation, transfers seamlessly to the Pro plan.

Additional benefits of the Pro plan include enhanced runtime logs retention (1 day versus 1 hour on Hobby), which significantly improves debugging capabilities in production. The 99.9% SLA provides contractual reliability guarantees absent from the Hobby tier. Concurrent build support (up to 12) eliminates queue times during active development. Preview deployment limits increase to 500 concurrent, enabling parallel feature development and review workflows that are essential for a team-based enterprise application.

2.4 Advantages and Risks

Advantages:

- Zero code changes required: immediate deployment after plan upgrade
- Preserves entire M4 Phase 3 CI/CD investment unchanged
- Lowest migration risk: no architectural refactoring needed
- Predictable base cost (\$20/month) with usage-based scaling
- 99.9% SLA with contractual reliability guarantees
- Enhanced observability: 24-hour log retention versus 1 hour on Hobby
- First-class Next.js support: built by the same team, optimized for App Router
- Instant rollback capability: every deployment is immutable and reversible
- Global edge network: 18+ points of presence for low-latency access
- Preview deployments: automatic per-PR deployment URLs for testing

Risks and Considerations:

- Vendor lock-in to Vercel platform: application optimized for Vercel-specific features
- Cost unpredictability at scale: usage-based billing can exceed \$200/month for high traffic
- Serverless cold starts: function initialization latency on infrequent endpoints
- Bandwidth overages: \$0.15/GB after 1TB monthly allocation
- Function duration limits: 800s max may constrain long-running AI operations

- Hobby plan is non-commercial: Pro required for production business applications anyway

3. Option 2: Alternative Hosting Platforms

3.1 Evaluation Criteria

Six alternative hosting platforms were evaluated against the specific requirements of DeepMindQ CRM: Next.js 16 support, PostgreSQL (Neon) connectivity, GitHub Actions CI/CD integration, production scalability, and rollback capability. Each platform was assessed on pricing model, serverless function limits (if applicable), operational complexity, and long-term viability. The evaluation draws on official documentation, pricing pages, community reports, and comparison articles published through mid-2026.

It is important to note that moving away from Vercel introduces a fundamental architecture shift: the application must be configured for container-based deployment using Next.js **output: "standalone"** mode, which is already set in the next.config.ts file. This means the Next.js server runs as a long-running Node.js process inside a Docker container rather than as individual serverless functions. While this eliminates the 12-function limit, it changes the operational model, scaling characteristics, and deployment workflow significantly.

3.2 Platform Comparison Matrix

The following comprehensive comparison matrix evaluates all six alternative platforms across the key decision criteria. Each platform was rated based on documented capabilities, current pricing, and community feedback as of August 2026. One critical finding during research was that AWS App Runner is being deprecated (closes to new customers April 30, 2026) and has been excluded from active consideration.

Platform	Est. Cost/mo	Next.js	Postgres	CI/CD	Rollback	Scalability
Vercel Pro	\$20-80	Native	Native	Built-in	Instant	Auto edge
AWS ECS/Fargate	\$60-120	Docker	Any TCP	GH Actions	1-click	Auto ALB
Azure Container Apps	\$5-38	Docker	Any TCP	GH Actions	Revision	KEDA auto
Railway	\$5-40	Nixpacks	1-click Neon	Auto/CLI	Full history	Limited

Platform	Est. Cost/mo	Next.js	Postgres	CI/CD	Rollback	Scalability
Render	\$25-32	Auto-detect	Managed +\$7	Auto-deploy	Manual	Manual scale
Fly.io	\$30-55	Docker	Own+\$38	fly-deploy	VM rollback	Multi-region

Table 5: Platform Comparison Matrix

3.3 Platform Deep Dives

3.3.1 AWS ECS/Fargate

AWS ECS with Fargate provides serverless container orchestration within the AWS ecosystem, eliminating the need to manage underlying EC2 instances. For DeepMindQ CRM, the application would run as a Docker container using Next.js standalone output mode, with an Application Load Balancer (ALB) routing traffic to the Fargate service. Neon PostgreSQL connects via standard TCP, and GitHub Actions handles the CI/CD pipeline with image pushes to Amazon ECR followed by ECS service updates.

The estimated monthly cost ranges from \$60-120 for a minimal production deployment: approximately \$38/month for a 1 vCPU / 2GB Fargate task running 24/7, \$22/month for the ALB, and additional costs for ECR storage, data transfer, and CloudWatch monitoring. Setup complexity is high, requiring configuration of ECR repositories, ECS clusters, task definitions, IAM roles, security groups, and Route53 DNS records. This infrastructure setup typically requires 2-4 days of dedicated DevOps work. A significant advantage is that AWS added 1-click rollback support in May 2025, and ECS supports canary and linear deployment strategies through CodeDeploy integration.

3.3.2 Azure Container Apps

Azure Container Apps (ACA) offers a compelling cost proposition with its generous free tier: 180,000 vCPU-seconds and 360,000 GiB-seconds per month, which is sufficient to cover low-to-moderate traffic applications entirely within the free allocation. The platform uses KEDA (Kubernetes Event-Driven Autoscaling) for automatic scaling, including scale-to-zero capability that eliminates costs during idle periods. The revision-based deployment system provides excellent rollback with traffic splitting support for canary deployments.

For DeepMindQ CRM, the container-based deployment would use Next.js standalone output in a Docker container pushed to Azure Container Registry. GitHub Actions integrates natively through Azure CLI and official Azure Actions. Neon PostgreSQL

connects via standard TCP. The primary trade-off is cold start latency of 1-3 seconds when the application scales from zero, which may affect user experience for the first request after idle periods. However, for an internal CRM application, this is generally acceptable. The estimated monthly cost of \$5-38 makes ACA the most cost-effective option for moderate traffic levels, though operational complexity is moderate and requires familiarity with Azure container networking concepts.

3.3.3 Railway

Railway provides the simplest developer experience among the alternatives, with built-in Nixpacks build detection for Next.js and a 1-click Neon PostgreSQL integration template. The platform automatically deploys on git push and provides full deployment history with instant rollback from the dashboard. Pricing starts at \$5/month minimum spend with usage-based compute charges of approximately \$20 per vCPU-month and \$10 per GB-month.

However, Railway has significant limitations for enterprise production use. A January 2026 analysis titled "Is Railway Reliable for Next.js in 2026?" documented systemic reliability concerns including unexpected service interruptions and insufficient scaling for production workloads. The Hobby plan offers no auto-scaling, limited to a single instance per service. For a mission-critical CRM application, these reliability concerns represent an unacceptable risk that disqualifies Railway from primary consideration, despite its excellent developer experience and Neon integration.

3.3.4 Render

Render offers a Heroku-like deployment experience with automatic Next.js detection, global CDN, managed TLS certificates, and predictable flat-rate pricing. The Standard web service plan at \$25/month provides 2GB RAM with always-on availability, making cost estimation straightforward. Render also offers managed PostgreSQL starting at \$7/month, though external Neon connectivity is fully supported via standard TCP connections.

The primary weaknesses are in rollback and scaling capabilities. Rollback is manual only, requiring deployment of a previous commit through the dashboard, with no automatic health-check triggered rollback or traffic splitting for canary deployments. Auto-scaling is not available on paid tiers (only scale-to-zero on the free tier), meaning instance count must be manually configured. Render restructured its pricing in April 2026, introducing workspace plans that increase costs for team-based usage. These limitations make Render suitable for small-scale production but less ideal for an enterprise CRM requiring robust deployment safety.

3.3.5 Fly.io

Fly.io specializes in multi-region deployment with Firecracker VM isolation, providing low-latency global access by deploying application instances close to users. The

platform shifted from pure usage-based pricing to subscription plans in 2026 (\$29/month Standard, \$149/month Premium) with additional usage charges on top. For DeepMindQ CRM, a 1GB shared CPU VM costs approximately \$5.70/month in usage, but the \$29/month plan is recommended for production reliability.

The managed PostgreSQL offering is expensive compared to Neon, starting at \$38/month for a basic 1GB instance with 1 replica. The Fly.io community has active discussions about migrating from Fly Postgres to Neon for cost reasons, which validates the architectural choice of using Neon as the primary database. Rollback is achieved by redeploying a previous VM image via CLI or dashboard, which is simpler than most alternatives but lacks the granular revision management of Azure Container Apps. The estimated total monthly cost of \$30-55 for application hosting plus Neon database makes Fly.io competitive but not the most cost-effective option.

3.4 Decision Matrix Summary

Priorit y	Platform	Best For	Monthly Est.	Risk Level
1st	Vercel Pro	Immediate deployment	\$20-80	Very Low
2nd	Azure Container Apps	Cost optimization	\$5-38	Low
3rd	AWS ECS/Fargate	AWS ecosystem	\$60-120	Medium
4th	Render	Simple production	\$25-32	Medium
5th	Fly.io	Multi-region needs	\$30-55	Medium
Avoid	Railway	Reliability concerns	\$5-40	High

Table 6: Platform Decision Matrix

4. Option 3: API Architecture Refactor

4.1 Refactor Approach Analysis

This option evaluates consolidating the 250 individual route.ts files into fewer handler files using Next.js catch-all route segments or grouped route patterns. The objective would be to reduce the total function count below the 12-function Hobby limit, or to a number acceptable for a chosen alternative platform. However, as this analysis will demonstrate, such a refactor represents a significant architectural regression with substantial risks that outweigh the perceived benefits.

4.2 Route Grouping Assessment

To achieve the 12-function limit on Hobby, the 250 routes would need to be consolidated into a maximum of 12 route handler files. This would require implementing a custom routing framework within each handler that dispatches requests based on URL path segments, HTTP method, and request parameters. The following table illustrates one potential grouping strategy and its associated complexity:

Consolidated Handler	Routes Absorbed	Estimated LOC	Auth Complexity
catch-all CRM routes	83 (companies, leads, contacts, opportunities)	2,500+	Very High
catch-all AI routes	35 (ai/, intelligence/)	1,800+	High
catch-all automation routes	18 (sequences, emails, cron)	1,200+	Medium
catch-all platform routes	20 (health, admin, settings, audit)	900+	Very High
catch-all data routes	15 (import, export, batches, seed)	800+	Medium
catch-all reporting routes	12 (reports, analytics, dashboard)	700+	High
catch-all knowledge routes	10 (knowledge, templates, playbooks)	600+	Medium
remaining 5 handlers	57 (misc single-route directories)	2,000+	Mixed
Total	250 routes into 12 handlers	10,500+	Extreme

Table 7: Route Consolidation Estimate

4.3 Impact Assessment

Authentication Impact:

The current architecture applies authentication at the route-handler level, allowing granular control over which endpoints require authentication, which require admin privileges, and which are publicly accessible. Consolidating 83 CRM routes into a single catch-all handler would require implementing a complex routing-aware authentication middleware within the handler, adding conditional logic that maps URL patterns to permission levels. This introduces a class of security bugs where a routing mistake could expose protected endpoints to unauthenticated access, or require authenticated users to re-authenticate for endpoints that should be public.

Testing Impact:

Each of the 18 existing smoke tests targets a specific API endpoint by its dedicated route path. Consolidation would require rewriting all test infrastructure to account for the new routing dispatch logic. More critically, unit testing individual route handlers becomes significantly harder when 20+ logical endpoints share a single file, as test isolation requires mocking the routing dispatch layer. Integration tests would need to verify both the dispatch logic and the handler logic, doubling the test surface area.

Middleware Impact:

Next.js middleware.ts (currently absent but planned) operates at the Edge level and can inspect request paths for authentication, rate limiting, and CORS decisions. A catch-all handler approach makes middleware less effective because the middleware must know the internal routing structure to make correct decisions, coupling the Edge middleware to the application routing implementation. This violates separation of concerns and creates maintenance burden whenever routes are added, modified, or removed.

Maintenance and Developer Experience:

The current structure allows developers to navigate directly to a specific route file based on the URL path: `/api/companies` maps to `src/app/api/companies/route.ts`. This intuitive mapping enables rapid feature development and debugging. Consolidation would require developers to first identify which catch-all handler contains a given endpoint, then locate the dispatch logic within a potentially 2,500-line file. Code review becomes harder, merge conflicts more frequent, and onboarding new developers significantly slower. For a 250-route enterprise application, this is a serious developer productivity regression.

4.4 Migration Risk

Risk Category	Severity	Probability	Impact
Authentication bypass via routing error	Critical	Medium	Data breach
Breaking existing API contracts	High	High	Client failures
Regression in smoke test coverage	High	High	Quality degradation
Merge conflicts in large handler files	Medium	Very High	Dev slowdown
Loss of Vercel per-function optimization	Medium	Certain	Performance hit
Difficulty adding new endpoints	Medium	High	Feature velocity drop
Incomplete migration leaving orphan routes	Medium	Medium	Deploy bloat

Table 8: Refactor Risk Assessment

The refactor would require changes to an estimated **250+ files** (all route handlers plus test files, type definitions, and import statements), with an estimated effort of **3-5 developer weeks** for a careful, tested migration. This effort produces zero new business capability and introduces significant regression risk, making it the least favorable option from both a cost-benefit and risk perspective. The refactor should only be considered if both Option 1 and Option 2 are deemed infeasible, which is not the case.

5. Recommendation

5.1 Primary Recommendation: Vercel Pro (Option 1)

The primary and immediate recommendation is to upgrade from Vercel Hobby to Vercel Pro. This recommendation is based on a weighted evaluation of five key factors: migration effort, architectural risk, cost predictability, time-to-deployment, and preservation of existing investments. Vercel Pro scores highest on four of these five factors, with the only caveat being vendor lock-in and cost unpredictability at very high traffic volumes.

The upgrade path is straightforward: create a Vercel Team, select the Pro plan, invite the existing project, and deploy. The entire process can be completed in under 30 minutes. The next deployment will succeed with all 250 serverless functions, the CI/CD pipeline will continue operating without modification, and the staging and production workflows will function as designed. This allows M4 Phase 3 to close immediately upon plan upgrade, unblocking progress toward M5 Business Logic and

Intelligence.

5.2 Implementation Path

Step	Action	Duration	Dependencies
1	Create Vercel Team account	5 minutes	Billing setup
2	Upgrade project to Pro plan	5 minutes	Step 1 complete
3	Verify environment variables transfer	10 minutes	Step 2 complete
4	Trigger staging deployment	5 minutes	Step 3 complete
5	Validate all 250 functions deploy	10 minutes	Step 4 complete
6	Run smoke tests against staging	10 minutes	Step 5 complete
7	Promote to production	15 minutes	Step 6 + approval
Total	Full deployment validation	~60 minutes	None

Table 9: Vercel Pro Implementation Timeline

5.3 Long-Term Strategic Consideration

While Vercel Pro is the recommended immediate solution, the project should evaluate container-based deployment as a Phase 5 infrastructure optimization initiative. Azure Container Apps offers the most compelling long-term value proposition with its generous free tier, KEDA-based auto-scaling, and revision-based deployment system. The migration path from Vercel to ACA is well-documented and can be executed incrementally using the existing **output: "standalone"** configuration already present in `next.config.ts`.

The recommended sequencing is: deploy immediately on Vercel Pro to unblock M4 Phase 3 closure and enable M5 progress, then evaluate container migration during Phase 5 when the application has established production traffic patterns that inform infrastructure optimization decisions. This two-phase approach balances urgency with strategic planning, ensuring the team is not forced into architectural decisions under deployment pressure.

5.4 Options NOT Recommended

Option 3 (API Architecture Refactor) is explicitly not recommended at this stage. Consolidating 250 well-organized route handlers into 12 catch-all files introduces critical security risks in authentication routing, degrades developer experience, doubles the test surface area, and requires 3-5 weeks of effort with zero business

value delivery. The current architecture follows Next.js best practices and should be preserved. Route consolidation should only be considered if a specific platform constraint demands it, which is not the case when either Vercel Pro or container-based hosting is available.

Railway is not recommended for production deployment due to documented reliability concerns in 2026, limited auto-scaling capabilities, and single-instance constraints on lower-tier plans. **AWS App Runner** is deprecated and should not be considered for new deployments. **Render** and **Fly.io** are viable alternatives but offer no compelling advantage over Vercel Pro for the immediate deployment need, while introducing additional operational complexity for container management.